

Laboratorio 4: Red

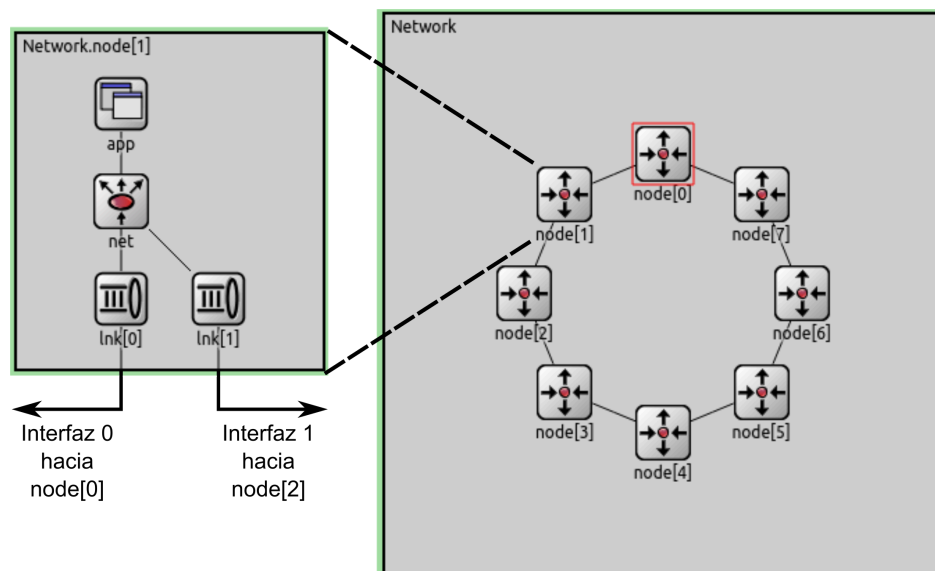
Cátedra de Redes y Sistemas Distribuidos 2026

Objetivos

- Leer, comprender y generar modelos de red en OMNeT++.
- Analizar tráfico de red bajo diferentes estrategias de enrutamiento.
- Diseñar e implementar soluciones de enrutamiento en OMNeT++ para distintas topologías.
- Redactar un artículo científico claro y ordenado en LaTeX con la información obtenida en los experimentos, la descripción del algoritmo de enrutamiento desarrollado y la comparación de resultados en los escenarios de prueba propuestos en el enunciado.
- Preparar un resumen tipo poster para presentar y defender en clase.

Modelo de Anillo

Se entrega como *kickstarter* un modelo de red en anillo¹ que consta de 8 nodos, cada uno con dos interfaces de comunicación *full-duplex*² con dos posibles vecinos.



¹ Las [topologías anillo](#) han sido, son y serán muy utilizadas en diferentes tipos de redes.

² [Full-duplex](#) quiere decir que el canal puede transmitir información en ambos sentidos al mismo tiempo.

Internamente, cada nodo cuenta con dos capas de enlace (lnk[0] y lnk[1], una con cada vecino), una capa de red (net) y una capa de aplicación (app). La capa de aplicación y la capa de enlace implementan generadores de tráfico y buffers respectivamente, tal como se usaron en el laboratorio de capa de transporte.

Conceptualmente, la novedad de este laboratorio es el rol de la capa de red, que **deberá decidir por qué interfaz enviar los paquetes que le llegan** ya sea desde la capa de aplicación superior o desde las capas de enlaces inferiores. En otras palabras, estaremos trabajando en el problema de **enrutar el tráfico** que confluye en este módulo de múltiples entradas y salidas.

En particular, la capa de red entregada en el kickstarter no reviste de mayor inteligencia. Cada paquete que ésta recibe es evaluado para determinar si el nodo local es el destino final del mismo. En caso de que lo sea, el paquete es enviado a la capa de aplicación local. En caso de que el paquete esté destinado a otro nodo se elige una interface para re-transmitirlo. La capa de red del kickstarter elige siempre la interface número 0 (toLnk[0]) que es la que envía el tráfico en sentido de las manecillas del reloj a lo largo del anillo hasta llegar al destino.

```
void Net::handleMessage(cMessage *msg) {
    // All msg (events) on net are packets
    Packet *pkt = (Packet *) msg;

    // If this node is the final destination, send to App
    if (pkt->getDestination() == this->getParentModule()->getIndex()) {
        send(msg, "toApp$o");
    }
    // If not, forward the packet to some else... to who?
    else {
        // We send to link interface #0, which is the
        // one connected to the clockwise side of the ring
        // Is this the best choice? are there others?
        send(msg, "toLnk$o", 0);
    }
}
```

Tarea Análisis

Se pide incluir en el modelo dado **todas las estadísticas necesarias** para analizar el rendimiento del sistema de enrutamiento planteado. En particular se sugiere incluir métricas de demora de entrega de paquetes, cantidad de saltos utilizados por cada paquete, utilización de los recursos de la red (buffers y enlaces), entre otras que el grupo crea conveniente.

Experimentos y Preguntas

1. **Caso 1)** Se deberá correr el modelo con las fuentes de tráfico configuradas (node[0] y node[2] transmitiendo datos a node[5]) y estudiar las métricas tomadas.
 - a. ¿Qué métricas se obtienen?
 - b. ¿Cómo es el uso de los recursos de la red?
 - c. ¿Se puede mejorar?
2. **Caso 2)** Asuma ahora que todos los nodos (0,1,2,3,4,6,7) generan tráfico hacia el node[5] con packetByteSize e interArrivalTime idénticos entre todos los nodos. Explore y determine a partir de qué valor de interArrivalTime se puede garantizar un equilibrio o estabilidad en la red. Justifique su respuesta.

```
Network.node[{0,1,2,3,4,6,7}].app.interArrivalTime = exponential(1)
Network.node[{0,1,2,3,4,6,7}].app.packetByteSize = 125000
Network.node[{0,1,2,3,4,6,7}].app.destination = 5
```

Tarea Diseño

Se solicita diseñar una estrategia de enrutamiento que supere la propuesta en el kickstarter. El grupo tiene total libertad para decidir qué información utilizará el nuevo algoritmo de enrutamiento y cómo cada nodo accederá a dicha información³. En esta parte de diseño deberán desarrollar un algoritmo de enrutamiento que no asuma una cantidad fija de nodos, osea que sea tolerante a cualquier cantidad de nodos del anillo.

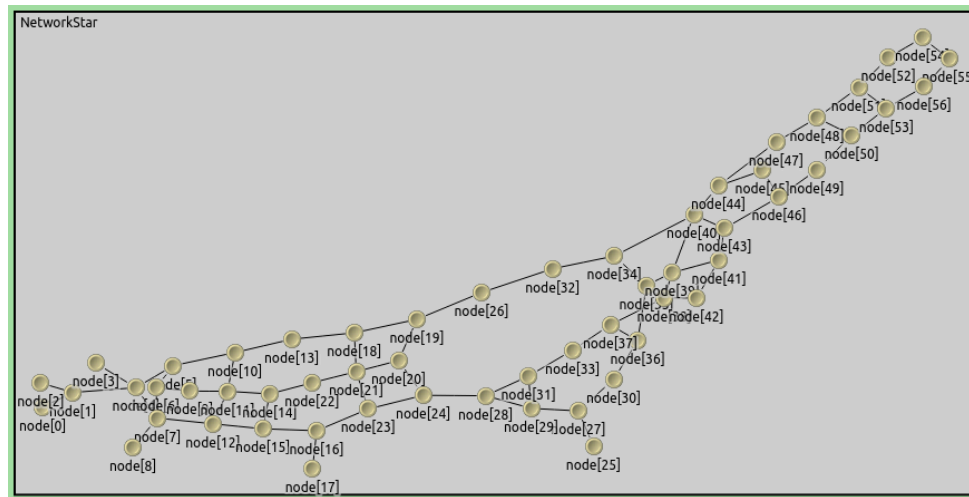
1. Evalúe y compare su estrategia con los casos 1 y 2 de la tarea de análisis⁴.
 - a. ¿Cuánto mejoran las métricas y por qué?
 - b. ¿Hay loops de enrutamiento?
 - c. Más allá de lo que llegó a implementar en el simulador, ¿ven alguna manera de mejorar más aún su diseño?
 - d. ¿Qué pasaría si la cantidad de nodos es mucho mayor y como funcionaria con respecto al algoritmo propuesto por la cátedra?
 - e. ¿Qué sucedería si se cae un nodo serían tolerantes a esa falla los algoritmos?

Tarea Estrella

³ Ni en la tarea de diseño ni en la tarea estrella se deberán usar funciones de simulación Omnet++ (i.e., cTopology, iteradores de cGate) para descubrir nodos vecinos. En cambio, se deberán aprovechar paquetes echo/hello como los que se usan en redes reales.

⁴ De ser necesario para un mejor funcionamiento de su algoritmo, reduzca el tamaño de los paquetes a 12500 Bytes y aumente su tasa de generación a exponential(0.1) (en el kickstarter están configurados 125000 Bytes y exponential(1) respectivamente).

Opcionalmente se pide diseñar una estrategia de enrutamiento más general. En particular, la misma debe adaptarse a cualquier topología y cantidad de interfaces por nodo⁵.



El grupo es libre de evaluar su algoritmo en las condiciones de tráfico que considere siempre y cuando justifique y discuta en el paper qué desafíos y problemas han debido superar para llegar al estado actual del algoritmo de enrutamiento propuesto.

Como banco de pruebas se provee un networkStar dentro archivo .ned del kickstarter. También se sugiere probar y comparar el algoritmo en la topología de anillo.

Requisitos de la Entrega

- El código debe ser claro y contener comentarios con detalles de lo implementado.
- Se deberá entregar en el repositorio un artículo científico (paper) de entre 6 y 8 páginas, le entregamos un template modelo en LaTeX que deben reutilizar. Aquí deben presentar los análisis de los experimentos y las respuestas a las preguntas de cada tarea. Además, deben incluir la explicación del algoritmo desarrollado, las decisiones de diseño y los resultados obtenidos, comparados con el algoritmo provisto por la cátedra en los escenarios de pruebas planteados.
- Se deberá entregar y presentar presencialmente un poster con el resumen del paper, ya que el día de clases posterior al deadline se realizará la muestra de todos los posters y los profesores recorrerán la muestra como evaluación final de este laboratorio (Se votarán los 3 mejores Posters y habrá premios).
- Primera entrega parcial deberán entregar el código fuente con tag *v0.0.1-EntregaParcial* y el paper completo hasta la sección **3.2 Caso 2: tráfico sostenido hacia node[5]** inclusive. Fecha límite **Lunes 01/06/2026 23:59h**.

⁵ El caso del anillo sólo considera dos interfaces por nodo, ahora esto se deberá generalizar a N interfaces. Inclusive, se deberá considerar que en un nodo determinado existan capas de enlace que no estén conectadas con nada (gates desconectadas). En particular, el código deberá testear qué interfaces se encuentran activas en el nodo.

- Entrega final deberán entregar el código fuente con tag *v0.0.2-EntregaFinal* , paper y poster completos. Fecha límite **Lunes 08/06/2026 23:59h.**